

Sprint 3 Demo Run Script — Dartly

Sprint 3 Demo Run Script

Dartly — ATS for Candidates | CS 490 Capstone

Quick Reference

Item	Value
Deployed URL	https://dartly-ten.vercel.app
Local URL	http://localhost:3000
Health probe	https://dartly-ten.vercel.app/api/health
CI workflow	.github/workflows/ci.yml
Deploy workflow	.github/workflows/deploy.yml
Test user A email	testuser@gmail.com
Test user A password	Testpass1!
Test user A ID	a818c364-412a-4545-8c88-f7b4cba05307
Test user B email	testuser2@gmail.com
Test user B password	Testpass1!
Test user B ID	77475f79-adb0-4de2-a61c-5ff34eb96ce7
Seed command	bun run db:clean && bun run db:seed
Test command	bun run test
CI pipeline	lint → type-check → test → build
Deploy pipeline	gated on CI → migrate deploy → Vercel prod → health probe

Seeded Data Summary

- **88 jobs** across 7 stages (Interested, Applied, Interview, Offer, Rejected, Ghosted, Archived)
- User A: 42 jobs, User B: 43 jobs
- **2 profiles** with full identity, experiences, educations, skills, and career preferences
- **Documents** — User A has 7 documents spanning RESUME / COVER_LETTER / OTHER types and DRAFT / READY / UPLOADED / ARCHIVED statuses with varied tags. User B has 3 documents for ownership-boundary checks.
- **Job-document links** seeded so both library-first and job-first flows have data on first open.
- **Company research** seeded on Bloom Health, InnovateTech (User A) and ScaleOps (User B).

- **Interview prep notes** (STAR Stories, Questions to Ask, Talking Points) seeded on the same jobs.
- **Activities and stage history** auto-generated per job based on stage transitions — feeds velocity and stage-conversion analytics.

Pre-Demo Checklist

- ☐ Confirm deployed app loads at `https://dartly-ten.vercel.app`
- ☐ Run `bun run db:clean && bun run db:seed` against the demo database — fresh seed data
- ☐ Login as User A (`testuser@gmail.com` / `Testpass1!`)
- ☐ Open GitHub Actions tab in a second tab — most recent CI + Deploy runs visible
- ☐ Pre-open three browser tabs: Dashboard, a Job Detail (Bloom Health), Document Library
- ☐ Open VS Code with the files listed below ready to show

A2 — Document Library End-to-End

File	Lines	What to Show
<code>src/app/(app)/documents/page.tsx</code>	164–220	Library page with filter bar, sort, list/card toggle
<code>src/components/documents/document-filter-bar.tsx</code>	161–228	Type filter, status filter, tag multi-select, search, archived toggle
<code>src/components/documents/document-filter-bar.tsx</code>	186–195	Sort options (Most recent / Name A–Z / Oldest first)
<code>src/app/api/documents/upload/route.ts</code>	40–188	Multipart upload, MIME allowlist (PDF), 10 MB max, magic-byte sniff, Zod validation, rate-limited
<code>src/app/(app)/documents/[id]/page.tsx</code>	560–585	Version selector + "Restore this version"
<code>src/services/documents.ts</code>	128–177	<code>duplicateDocument</code> , <code>renameDocument</code>
<code>src/services/documents.ts</code>	382–420	<code>archiveDocument</code> writes <code>previousStatus</code> ; <code>restoreDocument</code> reads it back
<code>src/components/documents/download-button.tsx</code>	160–185	<code>downloadDoc</code> handles both UPLOADED (signed URL) and generated markdown → PDF; filename includes version suffix

A3 — Job Context and Document Linking

File	Lines	What to Show
<code>src/app/api/jobs/[id]/documents/route.ts</code>	30–56	Link an existing version of a library document to a job

File	Lines	What to Show
<code>src/app/api/jobs/[id]/documents/[documentVersionId]/route.ts</code>	15–49	Unlink a specific version from a job
<code>src/services/documents.ts</code>	312–342	<code>linkDocumentToJob</code> — version-aware link creation, idempotent
<code>src/services/documents.ts</code>	344–379	<code>getDocumentsForJob</code> — surfaces <code>hasNewerVersion</code> so the UI can prompt an update
<code>src/app/(app)/dashboard/[jobId]/documents-section.tsx</code>	128–204	Link/Unlink UI
<code>src/app/(app)/dashboard/[jobId]/documents-section.tsx</code>	380–479	Linked-document list with <code>v{n}</code> and "Update to vN" hint

A4 — Company Research and Interview Prep

File	Lines	What to Show
<code>src/app/api/ai/research/route.ts</code>	12–62	Research endpoint: rate-limited (5/min), ownership check, Gemini call, <code>updateMany</code> scoped by <code>userId</code> for defense-in-depth
<code>src/services/ai.ts</code>	<code>generateCompanyResearch</code>	Gemini 2.5 Flash prompt with company, job title, description, optional user context
<code>src/app/(app)/dashboard/[jobId]/research-section.tsx</code>	108–195	Additional-context textarea (2k char cap), Generate, editable result with Save
<code>src/app/(app)/dashboard/[jobId]/prep-notes-section.tsx</code>	55–170	Three structured sections: STAR Stories, Questions to Ask, Talking Points
<code>prisma/schema.prisma</code>	98–101	<code>Job.companyResearch</code> , <code>prepNotesStar</code> , <code>prepNotesQuestions</code> , <code>prepNotesTalkingPoints</code>

A5 — Analytics

File	Lines	What to Show
<code>src/components/dashboard/metrics-panel.tsx</code>	307–358	Velocity card (last-30 vs prior-30, bar chart over 30 daily buckets)
<code>src/components/dashboard/metrics-panel.tsx</code>	360–385	Stage-conversion funnel (Interested → Applied → Interview → Offer)

File	Lines	What to Show
<code>src/components/dashboard/metrics-panel.tsx</code>	387–413	Average time in stage
<code>src/services/metrics.ts</code>	137–173	<code>computeVelocity</code> — UTC-day bucketing, deterministic across timezones
<code>src/services/metrics.ts</code>	175–210	<code>computeFunnel</code> — uses max stage rank seen, so terminal stages still count

B1 — Production Readiness Evidence

File	Lines	What to Show
<code>.github/workflows/ci.yml</code>	Full	Install → Prisma generate → Lint → Type-check → Test → Build
<code>.github/workflows/deploy.yml</code>	5–21, 43–46, 51–74	<code>workflow_run</code> gates on CI success, <code>prisma migrate deploy</code> , Vercel prod deploy, health probe with retries
<code>src/app/api/health/route.ts</code>	12–30	Unauthenticated GET, runs <code>SELECT 1</code> , returns 200 <code>{status:"ok", database:"up"}</code> or 503
<code>src/lib/logger.ts</code>	1–201	Winston, secret redaction, file transports in prod, exception/rejection handlers
<code>src/lib/api-error.ts</code>	1–32	<code>ApiError</code> class + <code>handleApiError</code> — consistent <code>{error: string}</code> body
<code>src/lib/api-wrapper.ts</code>	<code>withHttpLogging</code>	Wraps every API route for structured request/response logs

B1 — Tests Worth Citing

File	Line	Test Name	What It Proves
<code>src/tests/api/health.test.ts</code>	37	"returns 200 with status ok and database up when DB is reachable"	Health probe is a real readiness signal
<code>src/tests/api/health.test.ts</code>	53	"returns 503 with database down when DB is unreachable"	Health probe fails closed on DB outage
<code>src/tests/api/health.test.ts</code>	66	"logs the error when DB connectivity fails"	Centralized logging captures the failure
<code>src/tests/api/health.test.ts</code>	88	"returns 429 when rate limited"	Public endpoint is rate-limited
<code>src/tests/api/documents-upload.test.ts</code>	243	"returns 400 when MIME type is not allowed"	Upload validates MIME
<code>src/tests/api/documents-upload.test.ts</code>	254	"returns 400 when magic bytes don't match PDF"	Upload defends against MIME spoofing

File	Line	Test Name	What It Proves
src/tests/api/documents-upload.test.ts	271	"returns 400 when file exceeds 10MB"	Upload enforces size cap
src/tests/api/documents-upload.test.ts	303	"returns 500 and removes the storage object when DB write fails"	Atomic-ish: no orphan storage objects on failure
src/tests/api/documents-duplicate.test.ts	95	"scopes duplication to the authenticated user"	Cross-account isolation on a Sprint 3 endpoint
src/tests/services/metrics.test.ts	166	"counts applications inside the last 30 days only"	Velocity window is exact
src/tests/services/metrics.test.ts	210	"buckets applications into 30 daily bins"	Velocity chart binning is correct
src/tests/services/metrics.test.ts	275	"uses max history rank when current stage is terminal"	Funnel handles REJECTED/GHOSTED without losing prior progress
src/tests/services/metrics.test.ts	322	"computes average days for each transition (closed and current)"	Time-in-stage includes the still-open current stage

Test suite total: 36 files, 307 tests.

Phase A — Scripted Product Demo (12 minutes)

A1. Product Framing and Navigation Cohesion (1 min)

Step	Action	What to Show
1	Start on Dashboard	State the candidate goal: "apply, prepare, track, iterate"
2	Click into a Job Detail (Bloom Health)	Same global navigation, persistent sidebar, breadcrumb-style title; tabs are consistent (Overview, Timeline, Interviews, Follow-ups, Documents, Research, Prep Notes)
3	Click to Document Library from the sidebar	Same chrome, no visual jolt; loading skeleton appears briefly then the full library snaps in

A2. Document Library End-to-End Flow (4 min)

Step	Action	What to Show
1	Open /documents	Library grid/list — every row shows title, type, status, tags, updated date

Step	Action	What to Show
2	Apply filter	Set Type = "Cover Letter" → grid filters in real time; clear and set Status = "Ready" → only ready docs
3	Apply sort	Change sort from "Most recent" to "Name A-Z" → order updates; flip back
4	Upload a file	Click "Upload" → pick a PDF (have one ready) → validates MIME + magic bytes + 10 MB cap; success toast, new card appears
5	Open a document	Click "Resume - Cirrus Cloud" → editor view; switch to version selector (top right) → version history is visible; pick v1 → click "Restore this version"
6	Duplicate	Back to library → row action menu → "Duplicate" → new "Copy of ..." appears
7	Rename	Open the duplicate → rename inline → save → toast confirms
8	Archive	Row action menu → "Archive" on a READY doc → it moves to archived state; toggle "Show archived" on the filter bar to surface it
9	Restore	On the archived doc → "Restore" → it returns to its previous status (READY) — <i>not</i> a default DRAFT
10	Download / export	On a generated document → Download → markdown rendered to a PDF with the version number in the filename. On the uploaded PDF → Download fetches the signed URL

A3. Job Context and Document Linking UX (2 min)

Step	Action	What to Show
1	Navigate to a Job Detail (Bloom Health)	Click the "Documents" tab
2	Link an existing library document	"Link Document" picker → choose "Resume - Cirrus Cloud v1" → it appears in the Linked Documents list with a v1 badge
3	Unlink	Click "Unlink" on that row → it disappears; link is fully reversible
4	Re-link to a newer version	Edit "Resume - Cirrus Cloud" content elsewhere to create v2 → return → re-link → list shows v2 and the older link, if present, gets an "Update to v2" hint
5	Show the same flow from library	Back to <code>/documents</code> → open a document → linked jobs are visible from this side too

A4. Company Research and Interview Prep Workflow (2 min)

Step	Action	What to Show
1	Open Bloom Health Job Detail → "Research" tab	Existing seeded company-research note is visible immediately — the editor is populated, not empty
2	Trigger a regeneration with context	Type into "Additional context (optional)" — for example: "Focus on their React 19 migration and HIPAA scope" → click Generate → spinner → new content lands in the editable area
3	Edit and save	Tweak a line → Save → toast confirms; persisted to <code>Job.companyResearch</code>

Step	Action	What to Show
4	Click "Prep Notes" tab	Three structured sections render with seeded content: STAR Stories, Questions to Ask, Talking Points
5	Edit a section	Add a line to "Questions to Ask" → Save → toast confirms; persisted to <code>Job.prepNotesQuestions</code>
6	Emphasize: no context switch	All preparation artifacts live on the job itself, alongside the document links and timeline

A5. Analytics and "Ready for Production" Experience (2 min)

Step	Action	What to Show
1	Open Dashboard	Velocity card (last 30 days vs prior 30, with change-percent badge and a 30-day bar chart) and Stage Conversion funnel (Interested → Applied → Interview → Offer with rates)
2	Call out one actionable insight	E.g., "Interview rate is 35% and stage time is 9 days at Applied — next action is to push the 4 jobs sitting >14 days at Applied"
3	Show error/loading polish	Force a slow network in DevTools → skeletons appear (<code>src/components/ui/skeletons/dashboard-skeleton.tsx</code>); navigate to a non-existent job to show the global error boundary (<code>src/app/error.tsx</code>); show a toast on a save action
4	Resize to mobile	Layout reflows — sidebar collapses, cards stack, tap targets remain large

Phase B — Lightweight Evidence and Instructor Q&A (3 minutes)

B1. Deployment and Reliability Evidence (2 min)

Show one CI/CD run end-to-end:

Step	Action	What to Show
1	GitHub Actions tab	Open the most recent push to <code>main</code>
2	CI workflow	<code>ci.yml</code> — lint, type-check, test (307 tests), build all green; ~3–4 minute total
3	Deploy workflow	<code>deploy.yml</code> — gated by <code>workflow_run</code> on CI success; runs <code>prisma migrate deploy</code> , deploys to Vercel prod, then probes <code>/api/health</code>
4	Health probe in the deploy log	<code>curl --fail --retry 5 https://dartly-ten.vercel.app/api/health</code> returns <code>{status:"ok", database:"up"}</code>
5	Live health endpoint	Open <code>https://dartly-ten.vercel.app/api/health</code> in the browser → 200 JSON

Show centralized error logging/handling:

File	Lines	What to Show
<code>src/lib/logger.ts</code>	1–201	Winston configured with secret redaction, file transports in prod, exception + rejection handlers
<code>src/lib/api-error.ts</code>	1–32	<code>ApiError</code> class plus <code>handleApiError</code> — every API route returns a consistent <code>{error: string}</code> shape
<code>src/lib/api-wrapper.ts</code>	<code>withHttpLogging</code>	One wrapper used by every API handler; logs method, path, status, duration, and <code>requestId</code>
<code>src/app/api/ai/research/route.ts</code>	12–62	Concrete example: rate-limit, ownership check, scoped write, structured info log, error handler

Sanitized log sample (from a recent prod run):

```
2026-05-09T14:21:08.114Z INFO http req=req_8a3f path=/api/jobs/clxk0.../research method=POST s
2026-05-09T14:21:08.117Z INFO AI company research generated userId=a818c364... jobId=clxk0...
```

B2. Concise Q&A (1 min)

Q1: How did you keep UX coherent while adding many Sprint 3 workflows?

All Sprint 3 surfaces reuse the same primitives: the protected `(app)/layout.tsx` shell, a consistent tab layout on Job Detail, the shared toast and skeleton libraries (`src/components/ui/skeletons/*`), and a single `handleApiError` shape. New tabs (Research, Prep Notes) were added to the existing Job Detail tab strip — same nav, same chrome, no new navigation paradigms. The Document Library, Job Detail "Documents" tab, and document viewer all reuse the same filter and version components so the user sees identical metadata everywhere.

Q2: What was the highest risk area in deployment/hardening, and how did you reduce it?

The deploy → migrate → serve sequence. A failed migration after a green build would leave production in a partially-deployed state. Two mitigations: (1) `deploy.yml` is gated on `workflow_run: CI success`, so a broken build cannot trigger a deploy; (2) after `vercel deploy --prod`, we run `prisma migrate deploy` and then a `curl` health probe against the stable alias `https://dartly-ten.vercel.app/api/health` with five retries. The health endpoint runs `SELECT 1` against Prisma, so a 200 is a real readiness signal — not just "the build uploaded."

Q3: What would you improve next if given one additional sprint?

Three items in order: (1) Replace the in-memory rate limiter with a Redis-backed one so it survives across serverless instances; (2) add Playwright end-to-end tests for the document upload → version → link flow — currently we have unit tests for every layer but no full-stack browser test; (3) move AI generation to a background queue with streaming responses so the Research and Resume tabs do not block the UI on slow Gemini calls.

Scoring Rubric Quick Reference

Category	Points	Demo Coverage
Setup readiness and start on time	3	Pre-Demo Checklist
Demo flow preparedness and pacing	4	Phase A scripted timing
Completion within slot	3	Phase A 12 min + Phase B 3 min
Document Library end-to-end flow	15	Section A2
Job-context linking and visibility	10	Section A3
Company research + interview prep	10	Section A4
Analytics storytelling and clarity	10	Section A5
Required Sprint 3 workflows demonstrated	20	Sections A2–A5
Data persistence and state consistency	10	Save → reload checks throughout
Deployment / CI-CD / health check evidence	6	Section B1
Reliability and error handling evidence	4	Section B1 — logger + ApiError
Build/test evidence quality in CI/CD	5	Section B1 — CI run, 307 tests
Total	100	

Failure Mode Checklist

- ☐ Deployed URL serves traffic immediately when the professor opens it
- ☐ All Sprint 3 workflows demonstrated end-to-end (A2–A5)
- ☐ Document upload + version + duplicate + archive/restore all shown live
- ☐ At least one document linked, unlinked, and re-linked to a job in front of the audience
- ☐ Research and Prep Notes tabs show seeded content on first open (no empty states)
- ☐ Velocity + stage-conversion analytics rendered with a stated user-action insight
- ☐ CI green and visible in GitHub Actions
- ☐ Deploy workflow with health probe shown (or live health endpoint hit)
- ☐ Centralized logger / `handleApiError` referenced with file paths
- ☐ Completed within the 15-minute slot